Assignment 1: G04

Members: claoli1357 (Claudio Alberto Lucas De Oliveira), ulja2020 (Uliana Kulebyakina)
Repository: claoli1357/assig_1_mod550_g5 (private), copies: ulja2020/Assignment-0 (`Assignment-1/`)
Delivered: 06.10.2026 22:42, in time (the two README commits of 07.10 are not graded)
README: names and ids, no description, see task 0

101.5/120 + 5 bonus = 106.5 B

Task 0: (requirement, −3)

This is a new repository, not the one from assignment 0 (−3). Keep working in the assignment 0 repo from now on. The root `README.md` has your names and ids and nothing about what the repo holds. One line is enough.

Task 1: (requirement)

ok.
Suggestions: `drafts/` holds six old copies of the project. That is what git history is for, delete it from the tree. `data_tools_output_synthetic.py` is 3604 lines of generated data written as a python module: the seed already reproduces it, and if you want it on disk, a csv is the format for data. `test_model_tools.py` is extra, fine.

Part A: (66.25/70 pts)

Task 2: (4.75/5 pts)

Generator (2/2 pt): ok
Features (2/2 pt): ok
Metadata (0.75/1 pt): the README says the zones have "small effects" and that grain mass depends "partly" on ear length. The formula, the noise (sd 10 g) and the zone effects are not there; the coefficients only show up in task 9. Next: one table with how each feature is generated, what it depends on, and the noise.
Suggestions: a zone effect of 3 cm on a plant height spread of 18 cm is a group no clustering can find, and field_zone is never compared with your clusters.

Task 3: (4.5/5 pts)

Function (2/2.5 pt): the docstring of check_data_integrity (`data_tools.py`:262) says what it checks, not what it deliberately leaves out. The README says it receives a pandas DataFrame, it takes a dict.
Tests (2.5/2.5 pt): ok

Task 4: (8/10 pts)

Statistics (3/4 pt): describe_numerical_features (`stats_tools.py`:227) loops over NUMERICAL_FEATURES, the five maize column names fixed at line 8. The caller has to pass the columns.
Bins (1.25/1.5 pt): 25 because it makes features "comparable". Any fixed number does that, nothing ties 25 to 600 observations. `entropy_leaf_damage_vs_bins.png` is produced and never shown or discussed.
`plot_tools.py` (1.5/2 pt): FEATURE_DETAILS (line 22) and the zone list (line 38) are maize,

plot_clustering_comparison puts “600 maize plants” in the title (line 260), the nonsense and regression plots have grain mass and moisture fixed in their labels. Part B paid for it: five plot functions had to be rewritten in `project_2_b/code.py`. Next: labels and units as arguments.
`code.py` (1.25/1.5 pt): field_zone gets a frequency chart, but the README never says why it cannot have a mean, a std or a histogram.
Tests (1/1 pt): ok

Task 5: (4.75/5 pts)

Rescaling (0.75/1 pt): you say what entropy ignores, shift and scale. Not what it sees that the std does not: the shape, how the values spread over the bins.
The other four (4/4 pt): ok

Task 6: (5/5 pts)

ok.
Suggestions: `code.py` prints ear length vs grain mass, inf one way and 0.066 the other. That is your clearest asymmetry and it is not in the README.

Task 7: (4.75/5 pts)

Metric properties (1.5/1.5 pt): ok
Triangle inequality (1.5/1.5 pt): ok. Say which features P, Q and R are, the README gives three numbers and no names.
Example on your data (1/1 pt): ok
When KL is right (0.75/1 pt): Euclidean, Manhattan, Minkowski and Mahalanobis measure the distance between points, not between distributions. Next: Jensen-Shannon, Wasserstein, Hellinger.

Task 8: (15/15 pts)

ok.
Suggestions: TASK8_FEATURES in `model_tools.py`:14 is a maize default inside a tool. Leave the default out and let `code.py` pass the features.

Task 9: (15/15 pts)

ok.
Suggestions: R2 0.796 against 0.802 is barely wrong, the task asks for a clearly wrong model. The residual correlation with leaf damage is what saves it.

Task 10: (4.5/5 pts)

Table (1.5/1.5 pt): ok
Tests (1.5/2 pt): the README says to run the suite “From the FINAL folder” (line 583). There is no FINAL folder in this repo, it is `project_2_a`.
Broken input and bad models (1.5/1.5 pt): ok

Part B: (38.25/50 pts)

Task 11: (3.25/5 pts)

Repository (1/2 pt): no citation for FAOSTAT, and the endpoint you give, `/api/v1/ping`, is a ping. The code downloads the bulk zip. Next: the FAOSTAT data endpoint or the faostat python client, and the citation FAO asks for.

Features and missing values (1.5/1.5 pt): ok
Question (0.75/1.5 pt): no clustering question, and nothing on how you judge clusters without labels.

Task 12: (35/45 pts)

Download (7.5/10 pt): the committed CSV is not what the function downloads; its own metadata says "The CSV was supplied by the student". Delete `data/` and run `code.py`: the bulk file writes Area Code (M49) as `'004`, and line 450 crashes with `could not convert string to float`. Strip the apostrophe on that line and it runs to the end on the real download, 4236 observations and every figure. Only your own metadata is saved, nothing returned by the source.
Re-run of part A (15/15 pt): ok
Sensible and bad runs (5/10 pt): `code.py` runs both, the README discusses one. The regression on the real data (baseline, sensible, wrong, test R2 0.76 against 0.71) is not in the README at all, the clustering only through the Area Code silhouette.
Discussion (7.5/10 pt): on entropy you say it is not a measure of importance and stop there. The question asks which features are informative. Nothing on whether any distribution is multimodal.

Bonus: (+5)

- pylint: 0 (9.63 and 8.92)
- pytest, pydocstyle, pycodestyle: 0 (the 13 tests pass, but pycodestyle reports 125 warnings)
- submit 1 day earlier: 0
- classes: +5 (the dataclasses in `model_tools.py`)

A note on AI use

My AI checker reports a relatively high level of AI involvement in part A of this delivery. Part A was meant to be done without AI: the statistics, the models, the tests and the discussion are the content of the course, and they are what you need to be able to write yourselves.

Nothing is deducted for it. But every line you deliver is a line you should be able to explain in the oral, and that holds for each one of you, not only for whoever wrote it.

Overall

A lot of work, and most of it right. The weak point is the one note_2 is about: the tools know they are looking at maize, and part B is where that bill came due. Go back to `plot_tools.py` and `stats_tools.py` and take every column name out. Then run pylint, pycodestyle and pydocstyle on both folders and fix what they report. In part B, write the README for what `code.py` actually computes: half of your real data results are only in the terminal.

Evaluating the nonsense clusters in two spaces, 0.54 where they were made and -0.03 where they should mean something, was exactly the right way to show what a good score is worth.